

Lab10

GET /products

GET http://localhost:8080/products

Params Body Headers Auth Vars Script Assert >> JSON Prettify

1 ...

```
1 - {
2   {
3     "id": "1",
4     "name": "iPhone 15 Pro (673380402-4 SEC 3)",
5     "category": "Electronics",
6     "brand": "Apple",
7     "stock": 50,
8     "price": 39900.0,
9     "discountType": "MEMBER",
10    "discountedPrice": 35910.0
11  },
12  {
13    "id": "2",
14    "name": "MacBook Air M3",
15    "category": "Electronics",
16    "brand": "Apple",
17    "stock": 20,
18    "price": 49900.0,
19    "discountType": "NONE",
20    "discountedPrice": 49900.0
21  },
22  {
23    "id": "3",
24    "name": "Samsung Galaxy S24",
25    "category": "Electronics",
26    "brand": "Samsung",
27    "stock": 30,
28    "price": 29900.0,
29    "discountType": "SEASONAL",
30    "discountedPrice": 23920.0
31  }
32 }
```

Response Headers² Timeline Tests {} JSON 200 OK 179ms 485

```
public Flux<Product> getAllProducts() {
    return client.get()
        .uri("/products")
        .retrieve()
        .bodyToFlux(Product.class);
}
```

POST /products

POST http://localhost:8080/products

Params Body Headers Auth Vars Script Assert >> JSON Prettify

```
1 - {
2   {
3     "id": "4",
4     "name": "iPhone 15 Pro (673380402-4 SEC 3)",
5     "category": "apple",
6     "brand": "Apple",
7     "stock": 50,
8     "price": 39900.0,
9     "discountType": "MEMBER",
10    "discountedPrice": 35910.0
11  }
12 }
```

Response Headers² Timeline Tests {} JSON 200 OK 4

```

        .bodyToMono(Product.class)

    }

    public Mono<Product> createProduct(Product product)
    {
        return client.post()
            .uri("/products")
            .bodyValue(product)
            .retrieve()
            .bodyToMono(Product.class);
    }
}

```

DELETE /products/{id}

The screenshot shows a Postman interface for a DELETE request. The URL is `http://localhost:8080/products/4`. The response is a 200 OK status with a response time of 17ms. The response body is empty.

หลังจากลบแล้ว

The screenshot shows a Postman interface for a GET request. The URL is `http://localhost:8080/products`. The response is a 200 OK status with a response time of 6ms and a body size of 485B. The response body is a JSON array of three product objects.

```

1  ...
2  {
3    "id": "1",
4    "name": "iPhone 15 Pro (673388402-4 SEC 3)",
5    "category": "Electronics",
6    "brand": "Apple",
7    "stock": 50,
8    "price": 39900.0,
9    "discountType": "MEMBER",
10   "discountedPrice": 35910.0
11 },
12 {
13   "id": "2",
14   "name": "MacBook Air M3",
15   "category": "Electronics",
16   "brand": "Apple",
17   "stock": 20,
18   "price": 49900.0,
19   "discountType": "NONE",
20   "discountedPrice": 49900.0
21 },
22 {
23   "id": "3",
24   "name": "Samsung Galaxy S24",
25   "category": "Electronics",
26   "brand": "Samsung",
27   "stock": 30,
28   "price": 29900.0,
29   "discountType": "SEASONAL",
30   "discountedPrice": 23920.0
31 }
32 ]

```

```

        .bodyToMono(Void.class);
    }

    public Mono<Void> deleteProduct(String id) {
        return client.delete()
            .uri("/products/{id}", id)
            .retrieve()
            .bodyToMono(Void.class);
    }
}

```

GET /products/category/{cat}

The screenshot shows a REST client interface with the following details:

- Request:** GET http://localhost:8080/products/category/apple
- Response:** A JSON object representing an iPhone 15 Pro.

Response JSON:

```

{
  "id": "4",
  "name": "iPhone 15 Pro (673380402~4 SEC 3)",
  "category": "apple",
  "brand": "Apple",
  "stock": 50,
  "price": 39900.0,
  "discountType": "MEMBER",
  "discountedPrice": 35910.0
}

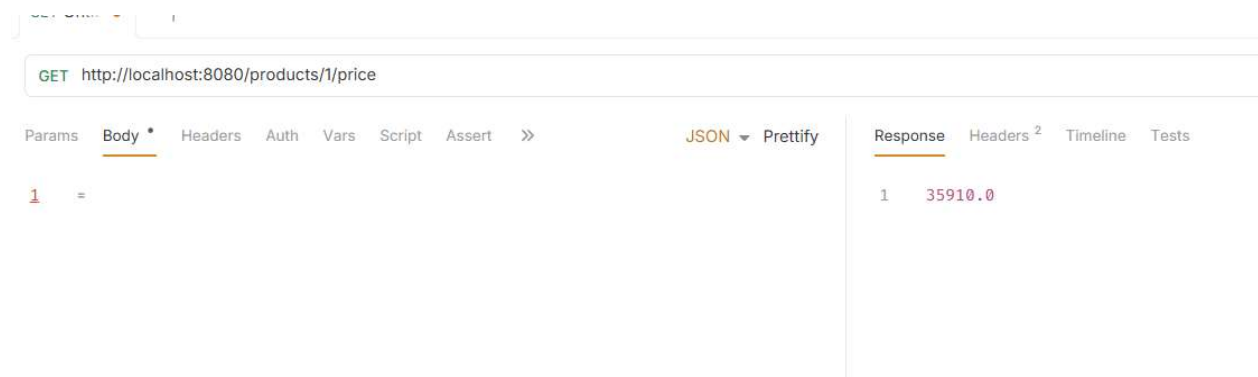
```

```

    public Flux<Product> getByCategory(String category) {
        return client.get()
            .uri("/products/category/{category}", category)
            .retrieve()
            .bodyToFlux(Product.class);
    }
}

```

GET /products/{id}/price



The screenshot shows a REST client interface with a request bar at the top containing 'GET http://localhost:8080/products/1/price'. Below the request bar are tabs for 'Params', 'Body', 'Headers', 'Auth', 'Vars', 'Script', 'Assert', and '>>'. The 'Body' tab is selected. To the right of the tabs are buttons for 'JSON' and 'Prettify'. On the far right, there are tabs for 'Response', 'Headers²', 'Timeline', and 'Tests'. The 'Response' tab is selected, showing a single response item with status '1' and value '35910.0'.

Response	Headers ²	Timeline	Tests
1 35910.0			

```
public Mono<Double> getDiscountedPrice(String id) {  
    return client.get()  
        .uri("/products/{id}/price", id)  
        .retrieve()  
        .bodyToMono(Double.class)  
        .doOnNext(price -> System.out.println("Price: " + price));  
}
```

1. Reactive Programming vs Blocking — ต่างกันอย่างไร

Reactive: ไม่ต้องรอผลลัพธ์ทันที แต่ส่งข้อมูลผ่าน Stream เมื่อข้อมูลพร้อม ทำให้รองรับงานพร้อมกันได้ดี

Blocking: ทำงานแล้วรอผลลัพธ์ก่อนจึงไปต่อ ทำให้ Thread วางรอ I/O เช่น Database หรือ HTTP

2. Mono vs Flux — ใช้กรณีไหน และทำไม

Mono: มีข้อมูล 0 หรือ 1 ค่า

ใช้กับการค้นหา Product หนึ่งรายการ

ใช้กับการบันทึก ลบ หรือคำนวณราคา

Flux: มีข้อมูล 0 ถึงหลายค่า

ใช้กับการดึง Product ทั้งหมด

ใช้กับการค้นหาตาม Category

3. WebClient — อธิบาย method chain `.get()`, `.uri()`, `.retrieve()`, `.bodyToMono/Flux()`

```
return client.get() // เลือก HTTP GET

    .uri("/products/{id}", id) // กำหนด URL และ path
variable

    .retrieve() // ส่ง request และรับ response

    .bodyToMono(Product.class);
```

- `.get()` กำหนด HTTP Method เป็น `GET`
- `.uri()` กำหนด URL ที่ต้องการเรียก
- `.retrieve()` เริ่มรับ HTTP response
- `.bodyToMono(Product.class)` แปลง response body เป็นข้อมูลเดียว
- `.bodyToFlux(Product.class)` แปลง response body เป็นข้อมูลหลายรายการ

4. Code ของ Repository, Service, Controller พร้อม comment อธิบาย

Repository

```
public Mono<Product> findById(String id) {  
  
    // ค้นหาจาก Map  
  
    Product product = store.get(id);  
  
  
    // ถ้าไม่พบคืน Mono.empty()  
    // ถ้าพบคืน Mono<Product>  
  
    return product == null  
        ? Mono.empty()  
        : Mono.just(product);  
}  
  
public Flux<Product> findAll() {  
  
    // แปลง Product หลายรายการจาก Map เป็น Flux  
  
    return Flux.fromIterable(store.values());  
}  
  
public Mono<Product> save(Product product) {  
  
    // บันทึกหรือแก้ไขข้อมูลใน Map  
  
    store.put(product.getId(), product);  
}
```

```
        // คืนข้อมูลที่บันทึกแล้ว 1 รายการ

        return Mono.just(product);
    }

    public Mono<Void> deleteById(String id) {

        // ลบข้อมูลตาม id

        store.remove(id);

        // การลบไม่มีค่าผลลัพธ์ จึงคืน Mono<Void>

        return Mono.empty();
    }

    public Flux<Product> findByCategory(String category) {

        // ดึงทั้งหมดแล้วกรองเฉพาะ category ที่ตรงกัน

        return findAll()

            .filter(product ->

                product.getCategory().equalsIgnoreCase(category)

            );
    }
}
```

Service

```
public Mono<Product> getById(String id) {  
    // เรียก Repository และจัดการกรณีไม่พบข้อมูล  
    return repository.findById(id)  
        .switchIfEmpty(  
            Mono.error(new RuntimeException("Product not found: " + id))  
        );  
}  
  
public Flux<Product> getAll() {  
    // ส่งต่อข้อมูลหลายรายการจาก Repository  
    return repository.findAll();  
}  
  
public Mono<Product> save(Product product) {  
    // ถ้ายังไม่มี id ให้สร้าง id ใหม่  
    if (product.getId() == null) {  
        product.setId(java.util.UUID.randomUUID().toString());  
    }  
  
    // เรียก Repository เพื่อบันทึก  
    return repository.save(product);  
}  
  
public Mono<Void> delete(String id) {  
    // เรียก Repository เพื่อลบข้อมูล  
    return repository.deleteById(id);  
}  
  
public Flux<Product> getByCategory(String category) {  
    // เรียก Repository เพื่อกรองตาม category  
    return repository.findByCategory(category);  
}  
  
public Mono<Double> getDiscountedPrice(String id) {  
    // map ใช้แปลง Product เป็นราคาหลังส่วนลด  
    return getById(id)
```

```
        .map(Product::getDiscountedPrice);
    }
}
```

Controller

```
@GetMapping("/{id}")
public Mono<Product> getById(@PathVariable String id) {
    // รับ request แล้วส่งต่อให้ Service
    return service.getById(id);
}

@GetMapping
public Flux<Product> getAll() {
    // คืบ Product หลายรายการ
    return service.getAll();
}

@PostMapping
public Mono<Product> save(@RequestBody Product product) {
    // รับ Product จาก request body แล้วบันทึก
    return service.save(product);
}

@DeleteMapping("/{id}")
public Mono<Void> delete(@PathVariable String id) {
    // ลบ Product ตาม id
    return service.delete(id);
}

@GetMapping("/category/{category}")
public Flux<Product> getByCategory(@PathVariable String category) {
    // คืบ Product ที่อยู่ใน category ที่ระบุ
    return service.getByCategory(category);
}

@GetMapping("/{id}/price")
public Mono<Double> getDiscountedPrice(@PathVariable String id) {
    // คืบราคาหลังส่วนลดของ Product
}
```

```
return service.getDiscountedPrice(id);  
}
```